

```
tile4 = board[(columnIndex, rowIndex + 3)]
if tile1 == tile2 == tile3 == tile4 == playerTile:
    return True
```

Next, we check for four-in-a-row tiles in a diagonal pattern going down and to the right; then we check for four-in-a-row tiles in a diagonal pattern going down and to the left:

```
for columnIndex in range(BOARD_WIDTH - 3):
    for rowIndex in range(BOARD_HEIGHT - 3):
        # Check for four-in-a-row going right-down diagonal:
        tile1 = board[(columnIndex, rowIndex)]
        tile2 = board[(columnIndex + 1, rowIndex + 1)]
        tile3 = board[(columnIndex + 2, rowIndex + 2)]
        tile4 = board[(columnIndex + 3, rowIndex + 3)]
        if tile1 == tile2 == tile3 == tile4 == playerTile:
            return True

        # Check for four-in-a-row going left-down diagonal:
        tile1 = board[(columnIndex + 3, rowIndex)]
        tile2 = board[(columnIndex + 2, rowIndex + 1)]
        tile3 = board[(columnIndex + 1, rowIndex + 2)]
        tile4 = board[(columnIndex, rowIndex + 3)]
        if tile1 == tile2 == tile3 == tile4 == playerTile:
            return True
```

This code is similar to the horizontal four-in-a-row checks, so I won't repeat the explanation here. If all the checks for four-in-a-row tiles fail to find any, the function returns `False` to indicate that `playerTile` is not a winner on this board:

```
return False
```

The only task left is to call the `main()` function:

```
# If this program was run (instead of imported), run the game:
if __name__ == '__main__':
    main()
```

Once again, we use a common Python idiom that will call `main()` if *fourinarow.py* is run directly but not if *fourinarow.py* is imported as a